

CA CO4 AT1 SET 1

NAME: SIVARANGJENII B

REG NO: 192521375

DEPARTMENT : B.TECH IT

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Industry Problem solving task
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A mobile device manufacturer observes noticeable lag when users switch rapidly between multiple applications. Analyze how L1, L2, and L3 caches and DRAM participate in storing and retrieving frequently accessed application data. Based on latency, capacity, hit rate, and bandwidth requirements, propose a suitable hierarchical memory organization that can reduce application-switching delays and improve overall system throughput.
2. A cloud service provider experiences increased latency when processing database queries from a large number of concurrent users. Analyze how cache hierarchy and virtual memory paging influence memory access performance, and compare their roles in reducing processor-memory latency. Recommend suitable memory technologies and memory-management improvements that can minimize page faults, reduce access delays, and improve database throughput.
3. An autonomous vehicle continuously receives large volumes of sensor data from cameras, LiDAR, radar, and other real-time sensing systems. Evaluate the characteristics of SRAM, DRAM, and MRAM in terms of access latency, bandwidth, capacity, volatility, and power consumption. Based on the real-time processing requirements, design an appropriate memory hierarchy that can provide high data-transfer bandwidth while minimizing access latency and supporting reliable sensor-data processing.
4. An edge-based AI device must load machine-learning models quickly while operating under strict power and storage constraints. Analyze the characteristics of NAND Flash, DRAM, and RRAM with respect to access speed, storage capacity, persistence, power consumption, and suitability for AI workloads. Recommend a hierarchical memory organization that enables fast model loading, efficient intermediate-data processing, reduced energy consumption, and improved overall inference performance.
5. A high-performance gaming console experiences frame drops and noticeable delays when loading large game environments containing high-resolution textures, audio, and other assets. Analyze the interaction between L3 cache and DRAM during the retrieval and processing of frequently accessed game data, considering their latency, capacity, and bandwidth. Propose suitable memory-hierarchy enhancements that can reduce data-access bottlenecks, increase memory throughput, and provide smoother frame rendering during large scene transitions.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 5	/ 100

Course Coordinator

Faculty Incharge

1. Mobile Device Manufacturer – Reducing Application-Switching Delay

Introduction

A mobile device may run several applications at the same time, such as a browser, music player, messaging application, camera, and background services. When a user switches rapidly between applications, the processor must retrieve instructions and data belonging to different applications.

If the required data is not available in the processor cache, the CPU has to access DRAM. DRAM has considerably higher latency than L1, L2, and L3 cache. Repeated DRAM accesses can therefore cause noticeable lag during application switching.

The solution is to design an efficient **hierarchical memory system** in which frequently accessed data is kept as close to the processor as possible.

Role of L1, L2, L3 Cache and DRAM

L1 Cache

L1 is the smallest and fastest cache.

It is normally divided into:

- Instruction cache
- Data cache

It stores the instructions and data that the processor is currently using.

Advantages:

- Extremely low latency
- Very high bandwidth
- High probability of finding recently used data

Limitation:

- Very small capacity

Therefore, L1 cannot store the complete working data of multiple applications.

L2 Cache

L2 is larger than L1 but slightly slower.

It acts as the next level when the required information is not found in L1.

For example:

CPU → L1 Miss → L2 Search

If the required data is found in L2, the processor does not have to access the slower DRAM.

L3 Cache

L3 is larger than L1 and L2 and generally has higher latency than both. In processors that include L3, it can serve as a larger shared cache.

During application switching, L3 can retain useful data belonging to applications that are not currently executing.

For example:

Application A → frequently used data

Application B → frequently used data

Application C → frequently used data

↓

L3

This can reduce the amount of data that must be retrieved from DRAM when switching between applications.

DRAM

DRAM acts as the main working memory.

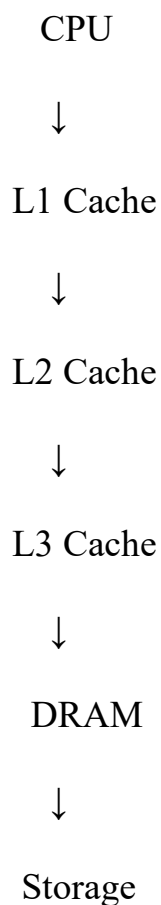
It has:

- Much larger capacity than cache
- Higher latency than cache
- High bandwidth
- Higher access energy than cache accesses

The operating system keeps active applications and their working data in DRAM.

Memory Access Process

The processor generally searches the hierarchy from the fastest level toward slower levels.



If data is found in L1, access is very fast.

If it is not found in L1 but is found in L2, the L2 access is required.

If it is not found in L1 or L2 but is found in L3, the processor uses L3.

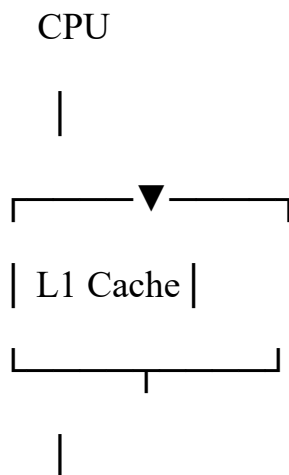
If it is absent from all cache levels, the processor must access DRAM.

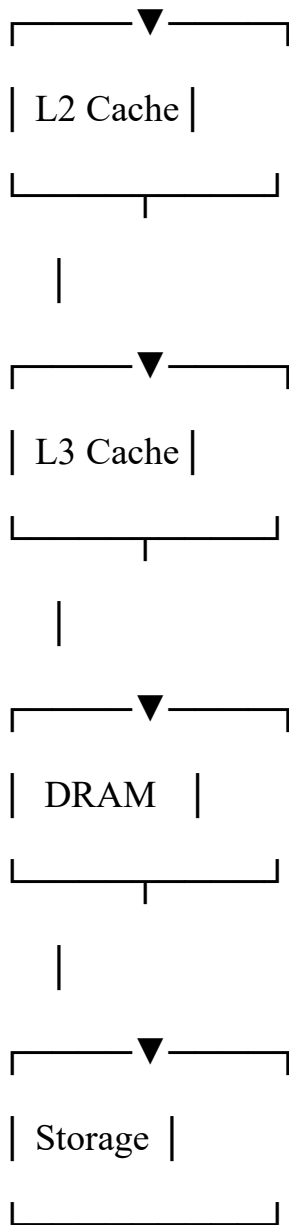
Comparison

Feature	L1	L2	L3	DRAM
Capacity	Very small	Small/Medium	Large	Very large
Latency	Very low	Low	Higher	High
Speed	Highest	Very high	High	Lower
Bandwidth	Very high	High	High	High
Cost per bit	Highest	High	Moderate	Lower
Main purpose	Current data	Recently data	used Larger working set	shared Main memory

Proposed Memory Hierarchy

A suitable organization is:





Improvements

1. Use sufficiently large L2/L3 caches.
2. Maintain high cache hit rates.
3. Use high-bandwidth mobile DRAM.
4. Improve cache replacement policies.
5. Keep frequently accessed application data in cache.
6. Avoid unnecessary movement of data between DRAM and storage.

7. Use efficient operating-system memory management.

Conclusion

The best solution is not to replace DRAM with cache completely because cache is expensive and has limited capacity. Instead, a **multi-level memory hierarchy** should be used.

L1 provides extremely fast access, L2 provides additional fast storage, L3 reduces expensive DRAM accesses, and DRAM provides large working memory. This organization reduces application-switching latency and improves overall system throughput.

2. Cloud Service Provider – Database Query Latency

Introduction

Cloud servers frequently handle thousands or millions of database requests from concurrent users. Database queries may repeatedly access the same instructions, indexes, records, and intermediate results.

When frequently required data is found in cache, the processor can access it quickly. However, cache misses and virtual-memory page faults can cause significant delays.

Therefore, both **cache hierarchy** and **virtual memory management** have an important effect on database performance.

Cache Hierarchy

The cache hierarchy is:

CPU

↓

L1 Cache



L2 Cache



L3 Cache



DRAM



Storage

Frequently used instructions and data are stored in cache.

For example, if a database repeatedly accesses the same index, some of the associated data may remain in cache.

Cache Hit

A **cache hit** occurs when the required data is found in the cache.

CPU → Cache → Data found

This results in low access latency.

Cache Miss

A **cache miss** occurs when the required data is not available at the searched cache level.

CPU → Cache → Data not found → Lower memory level

The processor then searches lower levels or main memory.

Virtual Memory

Virtual memory allows a system to provide each process with a large virtual address space.

The virtual address space is divided into **pages**, while physical memory is divided into **frames**.

The operating system maps virtual pages to physical frames.

When the required page is already in DRAM, access can continue normally.

If it is not present, a **page fault** occurs.

CPU



Virtual Address



Page Table



Page available in DRAM?



YES

NO



Access data Page Fault



Storage access



Load page



DRAM

Storage access is much slower than DRAM, so excessive page faults can significantly reduce database performance.

Cache vs Virtual Memory

Feature	Cache Hierarchy	Virtual Memory
Main objective	Reduce memory-access latency	Extend memory/address space
Managed mainly by	Hardware	Operating system + hardware
Fastest storage	L1 cache	DRAM for resident pages
Failure/miss	Cache miss	Page fault
Storage access	Usually not required	Required during page fault
Performance impact	Usually smaller	Can be very large
Main benefit	Faster CPU data access	Better memory utilization

Recommended Memory Technologies

1. High-performance DRAM

The server should have sufficient high-bandwidth DRAM.

More RAM allows more database pages to remain in physical memory and reduces paging.

2. Large Cache

A suitable L3 cache can help frequently accessed instructions and data.

3. NVMe SSD

If a page fault occurs, fast NVMe SSD storage can reduce the time needed to retrieve the page compared with slower storage.

However, the preferred solution is still to **avoid unnecessary page faults**.

Memory Management Improvements

Increase Physical RAM

If the database working set is larger than available physical memory, frequent paging can occur.

Increasing RAM allows more database pages to remain resident.

Database Buffer Pool

Database systems commonly maintain a memory area called a buffer pool.

Frequently accessed database pages can remain in memory.

This reduces repeated storage accesses.

Query Optimization

Poorly designed queries can cause unnecessary data accesses.

Using:

- Proper indexes
- Query optimization
- Efficient joins
- Appropriate data structures

can reduce memory requirements.

Reduce Page Faults

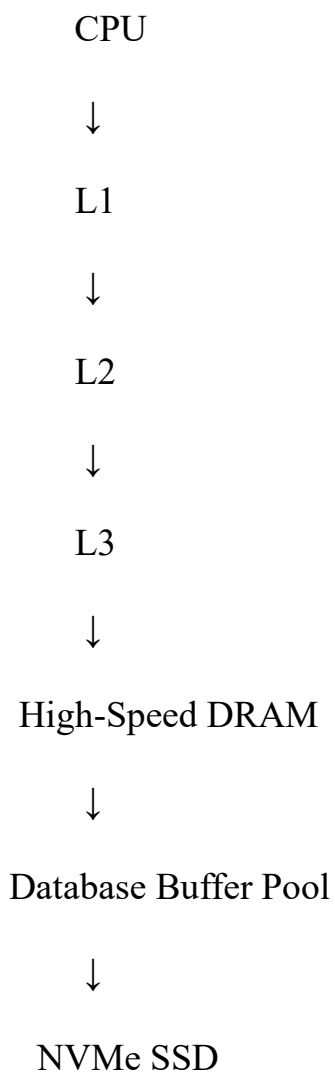
The operating system should efficiently manage page allocation and replacement.

Techniques such as:

- Appropriate page replacement
- Prefetching
- Avoiding unnecessary swapping
- Efficient memory allocation

can improve performance.

Proposed Architecture



Conclusion

Cache hierarchy and virtual memory solve different problems.

Cache hierarchy primarily reduces processor-memory latency, whereas virtual memory provides a larger logical memory space and manages physical-memory resources.

For cloud database systems, the best approach is to use large and fast caches, sufficient high-bandwidth DRAM, optimized database buffer pools, efficient queries, and fast NVMe storage as a secondary layer. Most importantly, minimizing cache misses and page faults will improve database throughput.

3. Autonomous Vehicle – SRAM, DRAM and MRAM

Introduction

An autonomous vehicle continuously receives large volumes of information from:

- Cameras
- LiDAR
- Radar
- Ultrasonic sensors
- GPS
- Other vehicle sensors

This data must be processed in real time.

For example, a camera may continuously generate image frames while LiDAR generates three-dimensional environmental information. If memory cannot

transfer this information quickly enough, the vehicle's perception and decision-making systems may be delayed.

Therefore, the memory system must provide **low latency, high bandwidth, sufficient capacity and reliable operation.**

SRAM

SRAM is very fast memory.

It does not require periodic refreshing like DRAM.

Advantages

- Extremely low latency
- High bandwidth
- Good for processor caches
- Suitable for frequently accessed data

Disadvantages

- Expensive
- Low density
- Requires more physical area per bit

Therefore, SRAM cannot economically be used as the vehicle's main memory.

DRAM

DRAM provides much greater capacity than SRAM.

It is suitable for storing:

- Camera frames
- LiDAR data

- Radar information
- AI intermediate data
- Operating-system data

Advantages

- High capacity
- High bandwidth
- Lower cost per bit than SRAM

Disadvantages

- Higher latency than SRAM
- Volatile
- Requires refresh operations

MRAM

MRAM is **non-volatile memory**.

It retains data even when power is removed.

Advantages

- Non-volatile
- Fast access
- Good endurance
- Low standby power
- Suitable for persistent information

Disadvantages

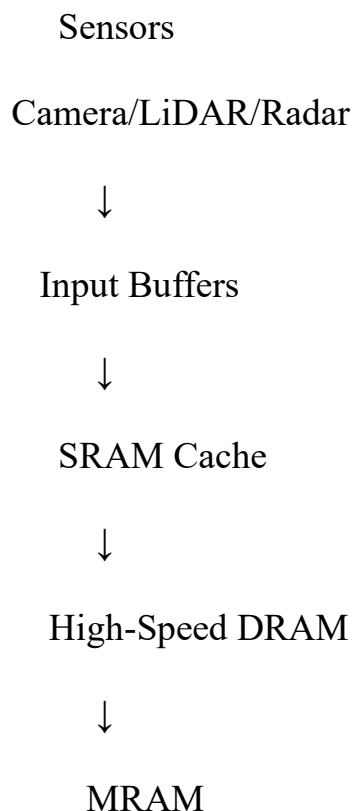
- Generally more expensive than conventional DRAM

- Lower capacity/density than mass-storage technologies

Comparison

Feature	SRAM	DRAM	MRAM
Latency	Very low	Low/Medium	Low
Capacity	Very low	High	Medium
Bandwidth	Very high	High	High
Volatile	Yes	Yes	No
Refresh required	No	Yes	No
Standby power	Can be significant	Requires refresh	Low
Main application	Cache	Main memory	Persistent memory

Proposed Hierarchy





Long-Term Storage

SRAM Usage

Immediately required sensor information should be placed in SRAM or processor cache.

For example:

Current camera frame



SRAM/cache



AI CPU/GPU

This reduces access latency.

DRAM Usage

Large sensor datasets should be placed in DRAM.

For example:

Camera frames

LiDAR point clouds

Radar information



DRAM



AI GPU

DRAM provides the capacity and bandwidth needed for these large datasets.

MRAM Usage

MRAM can store persistent information such as:

- Configuration information
- Important system states
- Selected persistent data
- Information that needs to survive power loss

Power Considerations

Autonomous vehicles have strict energy requirements.

Using SRAM for everything would consume excessive area and cost. Using DRAM for everything would increase latency.

Therefore, the correct solution is a balanced hierarchy.

Conclusion

The recommended design is to use **SRAM for extremely fast temporary data, DRAM for large high-bandwidth working datasets, and MRAM for persistent information.**

This hierarchy provides low latency for real-time processing while maintaining enough capacity for large sensor streams. It also improves reliability and energy efficiency.

4. Edge AI Device – NAND Flash, DRAM and RRAM

Introduction

Edge AI devices perform machine-learning inference locally instead of continuously sending data to a cloud server.

Examples include:

- Smart cameras
- Industrial AI devices
- Robots
- Drones
- Smart sensors
- Portable AI systems

These devices need to load machine-learning models quickly while operating with limited power, memory and storage capacity.

A suitable memory hierarchy can solve this problem.

NAND Flash

NAND Flash is a non-volatile memory.

It retains information even when power is removed.

It is commonly used for:

- Operating-system storage
- AI model storage
- Application storage
- Large datasets

Advantages

- High capacity

- Non-volatile
- Relatively low cost per bit
- Low standby power

Disadvantages

- Slower than DRAM
- Higher latency for random access
- Limited write endurance compared with RAM

Therefore, NAND Flash is best suited for **permanent model storage**, not continuous high-speed intermediate computation.

DRAM

DRAM is volatile but very fast compared with Flash storage.

It is suitable for:

- Loading AI models
- Input data
- Intermediate calculations
- Neural-network activations
- Temporary tensors

For example:

NAND Flash



AI Model



DRAM



AI Processor

Once the model is loaded into DRAM, the processor can access it much faster than directly accessing Flash repeatedly.

RRAM

RRAM, or Resistive RAM, is a non-volatile memory technology.

It can provide:

- Fast access
- High density
- Non-volatility
- Low-power operation

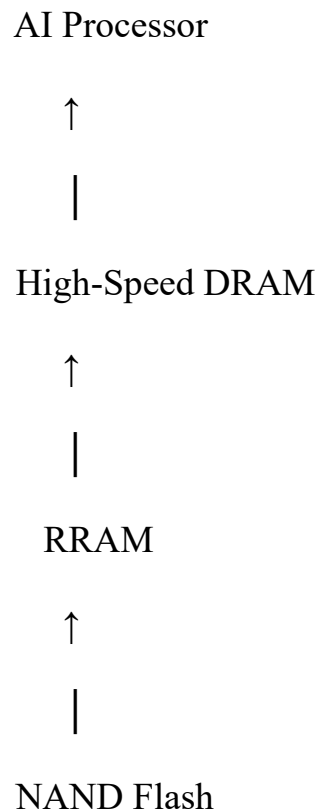
RRAM is particularly interesting for AI applications because emerging memory-computing architectures can potentially perform some AI operations close to or within the memory itself.

Comparison

Feature	NAND Flash	DRAM	RRAM
Speed	Lower	Very high	High
Capacity	Very high	High	Medium/High
Volatile	No	Yes	No
Persistence	Excellent	No	Excellent

Feature	NAND Flash	DRAM	RRAM
Power	Low standby	Higher active power	Potentially low
AI use	Model storage	Active processing	Persistent/AI-oriented memory

Proposed Hierarchy

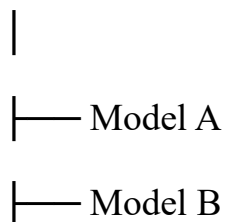


NAND Flash Layer

Large machine-learning models can be permanently stored in NAND Flash.

For example:

NAND Flash



└─ Model C

RRAM Layer

Frequently used model information or persistent AI parameters can potentially be placed in RRAM.

DRAM Layer

The currently running model should be transferred to DRAM.

The processor can then access model parameters and intermediate data much faster.

Model Loading Process

AI Model



NAND Flash



Fast transfer



DRAM



AI Accelerator



Inference

If the model has to be loaded repeatedly from slower storage, startup time increases.

Keeping frequently used models or important parameters in faster memory can reduce this delay.

Power Reduction

The system can reduce power consumption by:

1. Avoiding unnecessary Flash accesses.
2. Keeping frequently used data in faster memory.
3. Powering down unused memory regions.
4. Using low-power RRAM where appropriate.
5. Reducing unnecessary movement of data.

Conclusion

A combination of **NAND Flash, RRAM and DRAM** provides a good balance.

- NAND Flash provides large persistent storage.
- RRAM provides fast non-volatile storage.
- DRAM provides high-speed temporary working memory.

This organization allows AI models to be stored permanently while enabling fast model execution and efficient intermediate-data processing.

5. Gaming Console – L3 Cache and DRAM

Introduction

Modern games contain very large amounts of data, including:

- High-resolution textures
- 3D models

- Audio
- Animation data
- Physics data
- Game maps
- Lighting information
- Character data

During a scene transition, the processor and GPU may need to retrieve a large amount of data quickly.

If the required data cannot be supplied fast enough, the result may be:

- Frame drops
- Stuttering
- Loading delays
- Reduced frame rate
- Poor gaming experience

L3 cache and DRAM play important roles in avoiding these problems.

L3 Cache

L3 cache is much faster than DRAM but much smaller.

It is useful for storing frequently accessed data.

For example, during a game, the processor may repeatedly access:

- Character information
- Physics calculations
- Game logic

- Frequently used textures
- Frequently accessed object data

Keeping such data in cache reduces repeated DRAM accesses.

DRAM

DRAM provides much greater capacity.

It can hold:

- Large textures
- Game assets
- Audio data
- World/environment information
- Working datasets

However, DRAM has higher access latency than cache.

L3 Cache and DRAM Interaction

The basic process is:

CPU/GPU



L1/L2 Cache



L3 Cache



DRAM



SSD/Storage

Suppose the processor needs a texture-related data item.

Case 1: L3 Hit

CPU → L3 → Data Found

The data can be retrieved relatively quickly.

Case 2: L3 Miss but DRAM Hit

CPU → L3 → Miss

↓

DRAM

↓

Data Found

This takes longer than an L3 hit.

Case 3: Data not in DRAM

The system may have to obtain it from storage.

CPU

↓

L3 Miss

↓

DRAM Miss

↓

SSD

↓

DRAM



CPU/GPU

This is much slower and can contribute to visible stuttering during scene transitions.

Comparison

Feature	L3 Cache	DRAM
Capacity	Smaller	Much larger
Latency	Lower	Higher
Bandwidth	Very high	High
Cost per bit	High	Lower
Purpose	Frequently accessed data	Large working data
Volatility	Volatile	Volatile

Proposed Improvements

1. Increase L3 Cache

A larger L3 cache can store more frequently accessed game data.

This can improve the cache hit rate.

2. Use High-Bandwidth DRAM

High-bandwidth DRAM allows large amounts of texture and environment data to be transferred quickly.

This is particularly important for modern high-resolution games.

3. Improve Memory Controller

An efficient memory controller can schedule memory requests efficiently and make better use of available DRAM bandwidth.

4. Improve Asset Streaming

Instead of loading an entire game environment at once, the system can load assets according to the player's movement and expected requirements.

For example:

Player approaching new area



System predicts required assets



Assets loaded into DRAM



Required data becomes available



Scene transition becomes smoother

5. Fast SSD

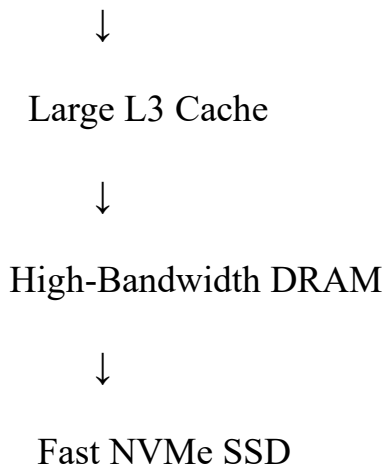
Modern consoles can use fast NVMe-based storage to reduce the time required to bring large assets into memory.

Proposed Hierarchy

CPU / GPU



L1 / L2



Effect on Gaming Performance

A well-designed hierarchy can:

- Increase cache hit rate
- Reduce DRAM accesses
- Increase effective memory bandwidth
- Reduce asset-loading delays
- Reduce frame drops
- Improve scene transitions
- Provide smoother gameplay

Conclusion

L3 cache should be used to store frequently accessed game data, while DRAM should provide the large working memory required for textures, audio and game environments. A larger L3 cache combined with high-bandwidth DRAM and fast SSD storage can reduce memory bottlenecks.

Therefore, the proposed hierarchy provides **low latency, high bandwidth and sufficient capacity**, resulting in smoother frame rendering and improved gaming performance.

